

Write a C program to simulate
a. Bankers' algorithm for the purpose of deadlock avoidance
Write safety algorithm, program and output

```
#include <stdio.h>

int main()
{
    int p, r, i, j, k;

    printf("Naysha Navin Shrivastava\n");
    printf("USN:1WA24CS184\n");
    printf("Enter number of processes: ");
    scanf("%d", &p);

    printf("Enter number of resource types: ");
    scanf("%d", &r);

    int allocation[p][r];
    int maximum[p][r];
    int need[p][r];
    int available[r];

    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < p; i++)
    {
        for(j = 0; j < r; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Maximum Matrix:\n");
    for(i = 0; i < p; i++)
    {
        for(j = 0; j < r; j++)
        {
            scanf("%d", &maximum[i][j]);
        }
    }
}
```

```
    }  
}
```

```
printf("\nEnter Available Resources:\n");  
for(i = 0; i < r; i++)  
{  
    scanf("%d", &available[i]);  
}
```

```
for(i = 0; i < p; i++)  
{  
    for(j = 0; j < r; j++)  
    {  
        need[i][j] = maximum[i][j] - allocation[i][j];  
    }  
}
```

```
printf("\nNeed Matrix:\n");  
for(i = 0; i < p; i++)  
{  
    for(j = 0; j < r; j++)  
    {  
        printf("%d ", need[i][j]);  
    }  
    printf("\n");  
}
```

```
int finish[p];  
int safeSequence[p];  
int work[r];  
int count = 0;
```

```
for(i = 0; i < p; i++)  
{  
    finish[i] = 0;  
}
```

```

for(i = 0; i < r; i++)
{
    work[i] = available[i];
}

while(count < p)
{
    int found = 0;

    for(i = 0; i < p; i++)
    {
        if(finish[i] == 0)
        {
            for(j = 0; j < r; j++)
            {
                if(need[i][j] > work[j])
                {
                    break;
                }
            }
        }

        if(j == r)
        {
            for(k = 0; k < r; k++)
            {
                work[k] += allocation[i][k];
            }

            safeSequence[count] = i;
            count++;
            finish[i] = 1;
            found = 1;
        }
    }
}

if(found == 0)

```

```

    {
        printf("\nSystem is NOT in Safe State\n");
        return 0;
    }
}

```

```

printf("\nSystem is in SAFE State\n");
printf("Safe Sequence:\n");

```

```

for(i = 0; i < p; i++)
{
    printf("P%d", safeSequence[i]);

    if(i != p - 1)
    {
        printf(" -> ");
    }
}

```

```

printf("\n");

```

```

return 0;

```

```

Naysha Navin Shrivastava
USN:1WA24CS184
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2

2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

System is in SAFE State
Safe Sequence:
P1 -> P3 -> P4 -> P0 -> P2

```

b. Deadlock Detection

Write (several instance Bankers Algorithm) algorithm, program and output

```
#include <stdio.h>

int main()
{
    int p, r, i, j, k;

    printf("Naysha Navin Shrivastava \n");
    printf("1WA24CS184 \n");
    printf("Enter number of processes: ");
    scanf("%d", &p);

    printf("Enter number of resource types: ");
    scanf("%d", &r);

    int allocation[p][r];
    int request[p][r];
    int available[r];

    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < p; i++)
    {
        for(j = 0; j < r; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Request Matrix:\n");
    for(i = 0; i < p; i++)
    {
        for(j = 0; j < r; j++)
        {
            scanf("%d", &request[i][j]);
        }
    }
}
```

```
    }  
}
```

```
printf("\nEnter Available Resources:\n");  
for(i = 0; i < r; i++)  
{  
    scanf("%d", &available[i]);  
}
```

```
int finish[p];  
int work[r];  
int sequence[p];  
int count = 0;
```

```
for(i = 0; i < r; i++)  
{  
    work[i] = available[i];  
}
```

```
for(i = 0; i < p; i++)  
{  
    int flag = 0;  
  
    for(j = 0; j < r; j++)  
    {  
        if(allocation[i][j] != 0)  
        {  
            flag = 1;  
            break;  
        }  
    }  
}
```

```
if(flag == 0)  
    finish[i] = 1;  
else  
    finish[i] = 0;  
}
```

```

int found;

do
{
    found = 0;

    for(i = 0; i < p; i++)
    {
        if(finish[i] == 0)
        {
            for(j = 0; j < r; j++)
            {
                if(request[i][j] > work[j])
                {
                    break;
                }
            }

            if(j == r)
            {
                for(k = 0; k < r; k++)
                {
                    work[k] += allocation[i][k];
                }

                sequence[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
} while(found);

int deadlock = 0;

```

```
printf("\nExecution Sequence:\n");

for(i = 0; i < count; i++)
{
    printf("P%d", sequence[i]);

    if(i != count - 1)
    {
        printf(" -> ");
    }
}

printf("\n");

printf("\nProcesses in Deadlock:\n");

for(i = 0; i < p; i++)
{
    if(finish[i] == 0)
    {
        printf("P%d ", i);
        deadlock = 1;
    }
}

if(deadlock == 0)
{
    printf("No Deadlock Detected");
}

printf("\n");

return 0;
}
```


Naysha Navin Shrivastava

1WA24CS184

Enter number of processes: 5

Enter number of resource types: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 3

2 1 1

0 0 2

Enter Request Matrix:

0 0 0

2 0 2

0 0 0

1 0 0

0 0 2

Enter Available Resources:

0 0 0

Execution Sequence:

P0 -> P2 -> P3 -> P4 -> P1

Processes in Deadlock:

No Deadlock Detected

=== Code Execution Successful ===